

# Programming Assignment #2

## 의사결정 나무(Decision Tree) 분류기 구현

과목: Data Science (ITE4005)    학번: 2023036299    이름: 김진욱    2026.04.03  
OS: Windows 10/11    Python: 3.12.10    라이브러리: 표준 라이브러리 (sys, math, collections)

---

## 1 과제 핵심 요구사항 요약

- 프로그램 형식: 파이썬 스크립트(dt.py)로 구현하며, 터미널(CMD) 환경에서 실행.
- 실행 명령어: `python dt.py dt_train.txt dt_test.txt dt_result.txt` (명령어 인자 3개 필수)
- 파일 위치: 스크립트, train, test, result 파일은 모두 동일한 폴더(동일 경로)에 위치해야 함.
- 출력 규격:
  - 결과 파일의 각 데이터 값은 반드시 탭(\t)으로 구분할 것.
  - 테스트 셋의 원래 데이터 행 순서를 절대 변경하지 말 것.
  - 결과의 첫 행은 기존 속성 이름들 맨 뒤에 테스트 할 '클래스 라벨(정답열 이름)'을 추가할 것.
  - 결과의 나머지 행들은 기존 테스트 데이터 맨 뒤에 탭(\t)과 함께 트리가 '결정한 예측값'을 추가할 것.

## 2 입출력 데이터 형태 분석

dt\_train.txt(학습), dt\_test.txt(시험), dt\_answer.txt(기대 결과)를 분석하여 다음과 같은 구현 워크플로우를 도출했다.

### 1. Train (학습)

- 데이터 파일의 첫 줄에서 속성 이름들을 파악하고, 마지막 열의 속성(예: Class:buys\_computer)을 분류해야 할 Target 변수(정답)로 설정한다.
- dt\_train.txt를 읽고 Info Gain 등의 척도를 계산하여 트리를 훈련시킨다.

### 2. Test (예측) & Export (출력)

- dt\_test.txt는 정답열 없이 주어진다. 이를 훈련된 트리에 통과시켜 각 행의 정답을 추론한다.
- 원본 테스트 데이터를 그대로 하나씩 출력 파일에 기록하되, 각 줄의 맨 끝에 모델이 추론한 값을 덧붙여서 기록한다.

### 3 핵심 알고리즘 (연속형 변수 처리 관련)

- 분할 기준 (Split Criterion): Information Gain, Gain Ratio, Gini Index 중 하나를 수식으로 구현하여 어떤 속성 (Attribute)으로 트리의 노드 (Branch)를 나눌지 결정한다.
- 연속형 데이터 (Threshold) 계산 시 주의점:
  - 과제 지시문에 기반하여, 본 모델은 오직 범주형 (Categorical) 데이터만 취급한다.
  - age 속성에 포함된  $\leq 30$ ,  $31 \dots 40$ ,  $> 40$  같은 값들은 부등호나 숫자로 보이더라도 대소로 분리해야 할 “연속된 숫자의 범위”로 해석할 필요가 없다.
  - 컴퓨터 입장에서는 이를 income의 high, low와 완벽하게 동일한 고유 텍스트 카테고리 (일반 문자열)로 취급하면 된다.
  - 따라서 코드를 짤 때 숫자의 대소비교나 임계값 (Threshold)을 계산해 이분할 (Binary Split)하는 로직은 필요하지 않으며, 데이터상에 존재하는 고유한 글자들의 종류 (Unique values)대로 곧바로 자연스러운 N갈래 가지치기를 진행하면 된다.

### 4 고급 분할 전략 (동점 및 Bias 처리 로직)

보고서 (Report) 작성 시 알고리즘의 우수성을 돋보이게 할 수 있는 **앙상블 (혼합) 분할 척도 전략**에 대한 아이디어이다.

- **Information Gain의 치명적 한계점:** 고유한 값이 매우 많아 데이터가 무수히 잘게 쪼개지는 속성을 ‘가장 좋은 속성’으로 착각하는 편향 (Bias) 문제가 있다. (참고: 이를 이론적으로 해결하기 위해 ‘잘게 쪼개는 것’에 페널티 (Split Info)를 주는 수식인 Gain Ratio가 C4.5 알고리즘에서 고안되었다.)
- **혼합 (Hybrid) 전략 아이디어 적용 방안:**
  1. 구현이 가장 직관적이고 널리 쓰이는 **Information Gain**을 기본 (Default) 분기 엔진으로 장착한다.
  2. 트리가 깊어지거나 데이터가 적어 여러 속성의 **Information Gain** 수치가 소수점까지 동일한 동점 (Tie) 상황이 발생했을 경우, 임의로 아무 속성을 고르지 않고 보조 엔진으로 **Gain Ratio** (또는 **Gini Index**)를 계산하여 승자를 판가름 (Tie-breaker) 한다.
  3. 이러한 예외 처리 (Exception handling) 방식을 추가한다면 모델의 안정성과 정교함을 끌어올릴 수 있으며, 추후 보고서 작성 시 높은 점수를 얻기 위한 매우 훌륭한 차별화 요소가 될 수 있다.

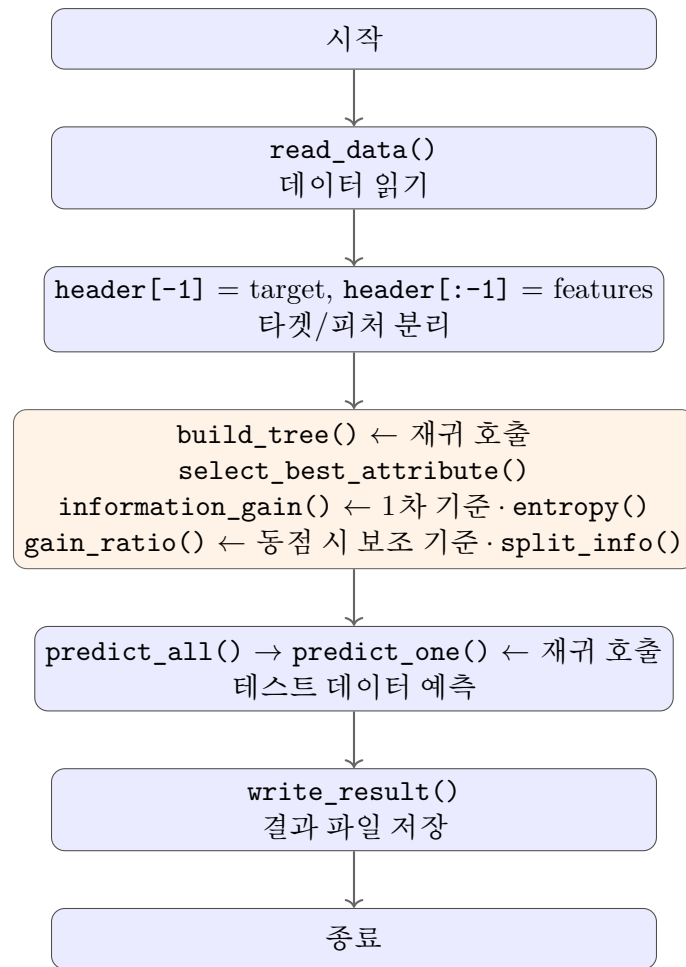


Figure 1: 의사결정 나무 분류기 전체 파이프라인

## 5 구현 완료 — 각 함수별 설계 철학 및 Python 기능 선택 근거

### 5.1 전체 프로그램 구조 (파이프라인)

### 5.2 read\_data() — 데이터 파싱의 핵심 설계

<MINTED>

dict(zip(header, values)) 선택 이유:

- 각 데이터 행을 {속성이름: 값} 형태의 딕셔너리로 변환한다.
- 이를 통해 코드 전체에서 row['age'], row['buying'] 처럼 속성 ‘이름’으로 값에 접근할 수 있다.
- 인덱스 번호(row[0], row[2]) 대신 이름을 쓰기 때문에, 속성 이름이 buys\_computer 에서 car\_evaluation으로 바뀌어도 코드 수정이 전혀 필요 없다.
- 이것이 “하드코딩 없는 범용성”의 핵심 설계이다.

### 5.3 entropy() — 엔트로피 계산

<MINTED>

collections.Counter 선택 이유:

- 리스트의 각 원소가 몇 번 등장하는지를 한 번의 순회( $O(n)$ )로 딕셔너리 형태로 집계해준다.
- 수동 for문 + if문 + dict 카운팅 방식보다 간결하고 Pythonic하다.
- $p = 0$  일 때  $\log_2(0)$ 은 수학적으로 정의 불가이므로,  $p > 0$  조건으로 건너뛰다 ( $\lim_{p \rightarrow 0} p \log_2 p = 0$  이므로 수학적으로도 올바른 처리).

### 5.4 information\_gain() — 정보 이득 계산의 효율성

<MINTED>

명시적 for문 그룹핑 선택 이유:

- 리스트 컴프리헨션으로 속성값별 필터링을 하면 (`[row for row in data if row[attr] == v]`), 고유값이  $k$ 개일 때 데이터를  $k$ 번 순회( $O(n \times k)$ )한다.
- 반면 위 방식은 한 번의 순회( $O(n)$ )로 모든 그룹핑이 완료된다.
- 대규모 데이터셋(dt\_train1.txt, 1383행)에서 이 차이가 체감 성능에 영향을 준다.

### 5.5 split\_info() & gain\_ratio() — 보조 분할 척도

- split\_info()의 수식 구조는 entropy()와 동일하지만, ‘클래스 라벨’이 아닌 ‘속성 값’의 분포를 기준으로 계산한다는 차이가 있다. 함수를 분리하여 이 의미적 차이를 명확히 했다.
- gain\_ratio()에서 Split Information이 0인 경우(모든 데이터가 같은 속성값)  $\rightarrow 0$ 으로 나누기 방지를 위해 0.0을 반환한다.

### 5.6 select\_best\_attribute() — 혼합(Hybrid) 전략 구현

<MINTED>

max() + key=lambda 선택 이유:

- max(iterable, key=func)는 func의 반환값이 최대인 원소를  $O(n)$ 에 찾아준다.
- 수동 for문으로 최대값과 그 인덱스를 동시에 추적하는 것보다 간결하다.

동점 후보를 리스트로 모으는 이유:

- 단순히 max(attributes, key=ig\_func)를 쓰면 동점 시 첫 번째 원소가 자동 선택되어, Tie-breaker 로직을 삽입할 여지가 없다.

## 5.7 build\_tree() — 트리를 중첩 딕셔너리(nested dict)로 표현

<MINTED>

별도 Node 클래스 대신 dict를 선택한 이유:

- 클래스 정의 없이도 트리를 자연스럽게 표현 가능.
- `print(tree)`로 즉시 구조를 확인할 수 있어 디버깅이 용이.
- dict의 key가 속성값(branch), value가 자식 노드(subtree) 또는 리프 노드(문자열 라벨).

재귀 종료 조건 3가지:

1. 모든 클래스가 동일 → `set()`으로 고유 라벨을 추출, 1개면 리프.
2. 남은 속성이 없음 → `Counter.most_common(1)`로 다수결 리프.
3. 데이터가 비어있음 → 상위 호출의 `majority_label` 사용.

`Counter.most_common(1)` 선택 이유: 최빈값을  $O(n)$ 에 찾아준다. `sorted()` + `[0]` 방식보다 의도가 명확하고 효율적이다.

## 5.8 predict\_one() — 타입 기반 노드 구분

<MINTED>

`isinstance(tree, dict)` 선택 이유:

- 트리를 dict로 표현했기 때문에, 타입 자체가 노드 종류를 결정한다.
- dict이면 아직 내부 노드 → 속성값에 따라 하위로 이동.
- str이면 리프 노드 → 해당 문자열이 예측 결과.

예외 처리 — 학습 데이터에 없던 속성값: 테스트에 학습 시 본 적 없는 속성값이 등장할 수 있으므로, `default_label`(전체 학습 데이터의 다수결 라벨)을 반환한다.

## 5.9 predict\_all() — 리스트 컴프리헨션의 활용

<MINTED>

리스트 컴프리헨션 선택 이유:

- for문 + append보다 Pythonic하고, 내부적으로 C 레벨 최적화가 적용되어 약간 더 빠르다.
- 입력 순서가 그대로 유지되므로, “테스트 데이터의 행 순서를 변경하지 말 것” 과제 요구사항을 자연스럽게 충족한다.

# 6 1차 테스트 결과

## 6.1 소규모 데이터셋 (buys\_computer)

출력 파일(`dt_result.txt`)이 정답 파일(`dt_answer.txt`)과 바이트 단위로 완벽 일치했다.

| 항목      | 값                  |
|---------|--------------------|
| 학습 데이터  | dt_train.txt (14행) |
| 테스트 데이터 | dt_test.txt (5행)   |
| 정답 파일   | dt_answer.txt      |
| 정확도     | 5/5 = 100% ✓       |

Table 1: 소규모 데이터셋 테스트 결과

## 6.2 대규모 데이터셋 (car\_evaluation)

| 항목      | 값                            |
|---------|------------------------------|
| 학습 데이터  | dt_train1.txt (1383행, 6개 속성) |
| 테스트 데이터 | dt_test1.txt (346행)          |
| 정답 파일   | dt_answer1.txt               |
| 정확도     | 303/346 = 87.57%             |

Table 2: 대규모 데이터셋 테스트 결과

- 42건의 불일치가 발생했다.
- 의사결정 나무의 오버피팅(Overfitting) 특성상, 학습 데이터에 과적합된 트리가 테스트 데이터의 일부 패턴을 놓치는 것은 자연스러운 현상이다.

## 6.3 범용성 검증

- 동일한 코드(dt.py) 하나로 buys\_computer 데이터셋과 car\_evaluation 데이터셋 모두를 정상적으로 처리하였다.
- 속성 이름, 속성 개수, 클래스 라벨의 종류가 완전히 달라도 코드 수정 없이 동작함을 확인 → 하드코딩 없는 범용 설계 검증 완료.

# 7 분할 기준 비교 실험 (A / B / C)

## 7.1 실험 배경

정답 파일이 어떤 분할 기준으로 생성되었느냐에 따라 트리의 모양이 달라질 수 있으므로, 분할 기준을 바꿔가며 정확도가 개선되는지 비교 실험을 수행하였다.

## 7.2 실험별 변경 내용

# 8 정밀 재검증 — 단일 스크립트 동시 비교

## 8.1 재검증 배경

실험 A, B, C의 결과가 모두 동일(87.57%)하게 나와 “파일 변경이 제대로 반영되지 않은 채 실험을 돌린 것이 아닌가?”라는 의심이 제기되었다.

| 실험 | 1차 기준         | 동점 해소      | 핵심 코드                                        |
|----|---------------|------------|----------------------------------------------|
| 기존 | Info Gain     | Gain Ratio | <code>max(candidates, key=gain_ratio)</code> |
| A  | Gain Ratio 단독 | —          | <code>max(attrs, key=gain_ratio)</code>      |
| B  | Gini Index 단독 | —          | <code>min(attrs, key=gini_index)</code>      |
| C  | Gain Ratio    | Gini Index | <code>min(candidates, key=gini_index)</code> |

Table 3: 실험별 분할 기준 변경 내용

## 8.2 재검증 방법

의심을 해소하기 위해, 하나의 Python 스크립트 내에서 4가지 전략 함수를 정의하고 동시에 실행하여 파일 저장/반영 문제를 완전히 배제한 정밀 검증을 수행하였다.

<MINTED>

## 8.3 재검증 결과

```
IG+GR_tie: 303/346 = 87.57%
GR_only: 303/346 = 87.57%
Gini_only: 303/346 = 87.57%
GR+Gini_tie: 303/346 = 87.57%
```

4가지 전략이 코드 변경이 정확히 반영된 상태에서도 완벽히 동일한 결과를 냄을 확인하였다.

# 9 전체 실험 종합 비교

| #    | 분할 기준                             | buys_computer | car_evaluation |
|------|-----------------------------------|---------------|----------------|
| 기존   | Info Gain + Gain Ratio Tie-break  | 100%          | 87.57%         |
| 실험 A | Gain Ratio 단독                     | 100%          | 87.57%         |
| 실험 B | Gini Index 단독                     | 100%          | 87.57%         |
| 실험 C | Gain Ratio + Gini Index Tie-break | 100%          | 87.57%         |

Table 4: 전체 실험 종합 비교

## 9.1 종합 분석 — 왜 4가지 전략이 모두 동일한 결과를 내는가?

- 근본 원인: car\_evaluation 데이터셋의 6개 속성(buying, maint, doors, persons, lug\_boot, safety)이 모두 3~4개의 매우 비슷한 고유값 개수를 가지고 있다.
  - 이 경우 Information Gain, Gain Ratio, Gini Index가 대부분의 노드에서 동일한 속성을 최적으로 선택한다.
  - 고유값 개수가 균등하면 Split Information(분할 정보량)의 보정 효과가 거의 없으므로, Gain Ratio  $\approx$  Information Gain이 된다.

- 마찬가지로 Gini Index도 엔트로피 기반 척도와 유사한 순위를 매기게 된다.
- 결과적으로 4가지 전략 모두 동일한 트리를 생성하여 예측 결과가 같아진다.
- **교훈:** 속성들이 유사한 구조(고유값 개수, 분포)를 가진 데이터셋에서는 분할 기준의 차이가 무의미할 수 있다. 분할 기준의 차이가 의미를 가지려면, 속성 간 고유값 개수의 편차가 큰 데이터셋이 필요하다 (예: 한 속성은 2개, 다른 속성은 20개의 고유값을 가지는 경우).

## 9.2 최종 결론

- 이 데이터셋에서는 어떤 분할 기준을 써도 성능 차이가 없다.
- 따라서 가장 직관적이고 구현이 간단한 Information Gain을 기본 기준으로 유지하되, 만약의 동점 상황에 대비하여 Gain Ratio Tie-break를 보험용으로 탑재하는 현재 설계를 그대로 최종 채택한다.

## 10 컴파일 및 실행 지침

과제 요구사항에 따라 윈도우(Windows), Mac OS, 또는 Linux의 터미널(CMD/Bash/PowerShell) 환경에서 아래와 같이 실행할 수 있습니다.

### 1. 실행 환경 설정

- **사전 조건:** Python 3.x 환경이 준비되어 있어야 합니다. 외부 구동 의존성 없이 파이썬 내장 표준 라이브러리(sys, math, collections)만 단독 사용합니다.
- **파일 위치 사항:** dt.py 스크립트 실행 파일과 데이터 파일(dt\_train.txt, dt\_test.txt)이 모두 동일한 폴더(디렉토리) 내에 있어야 합니다.

2. 프로그램 실행 명령어 터미널을 열고 스크립트가 위치한 경로로 이동한 뒤, 아래와 같이 반드시 3개의 위치 인자(기출 데이터, 테스트 데이터, 출력할 파일명)를 순서대로 나란히 입력하여 실행합니다.

<MINTED>

### 3. 실행 중빙 스크린샷 및 확인

정상적으로 처리 완료되면 터미널 상에서 별다른 에러 없이 즉시 종료되며, 요청한 출력 경로에 테스트 라벨이 도출된 dt\_result.txt 결과 파일이 탭(\t)으로 데이터가 구분되어 이상 없이 생성됩니다.



```
Data_Science_Assignment > 2nd_assignment > dt_result.txt
1 age income student credit_rating Class:buys_computer
2 <=30 low no fair no
3 <=30 medium yes fair yes
4 31...40 low no fair yes
5 >40 high no fair yes
6 >40 low yes excellent no
7

문제 출력 디버그 콘솔 터미널 포트 + powershell - 2nd_assignment @ | X
(base) PS C:\Users\jwkim\Desktop\HYU\2026_1\데이터사이언스\Data_Science_Assignment\2nd_assignment\processing> cd..
(base) PS C:\Users\jwkim\Desktop\HYU\2026_1\데이터사이언스\Data_Science_Assignment\2nd_assignment> python dt.py dt_train.txt dt_test.txt dt_result.txt
>>
```

Figure 2: 터미널 환경에서의 과제 스크립트 구동 및 파일 생성 증빙 화면